

INDIVIDUAL ASSIGNMENT 1

ML System Design Document

Telecom Customer Churn
A reproducible retention-risk system

Trịnh Đức Dương

Student ID: 25ms13290

Course: DDM501 - AI in DevOps, DataOps, MLOps

Submission: Individual written report

Revised: 6 October 2026

Companion GitHub repository

<https://github.com/TrinhDucDuong/ddm501-assignment-1-telco-churn-system-design>

Implementation snapshot: [290f2fd](#). The PDF is self-contained; code, data and execution evidence supplement the report.

Contents

Executive brief	3
1. Problem definition	4
2. Functional requirements and contracts	5
3. Non-functional and data requirements	6
4. Goals, metrics and acceptance criteria	7
5. High-level architecture	8
6. Trade-offs and selected decisions	9
7. Executed prototype and evaluation	10
8. Submission evidence and reproduction	11
References and artifact map	12

Executive brief

This document designs a customer churn risk system for a hypothetical subscription telecom operator. It ranks active customers for a retention team whose contact capacity is limited. The proposed production system combines daily batch predictions, data quality gates, a versioned training pipeline, controlled release and monitored feedback. A single supervised classifier is sufficient; an additional recommendation or uplift model is deliberately outside scope.

The business question is: which customers should receive a retention review today? The model estimates churn risk, while the retention agent chooses an appropriate action subject to consent and contact policies. Risk does not imply that an offer will change a customer's decision. A randomized pilot is required before claiming incremental retention or financial benefit.

The accompanying project runs on the publicly available IBM Telco Customer Churn sample [1]. It contains 7,043 fictional customer records and an observed churn label. The source has no dated observation window or individual event history, so this prototype demonstrates engineering and offline discrimination, not a validated prediction of future churn. A production implementation would create dated monthly snapshots and a next-30-day label before any launch.

The deliverables are a runnable baseline, tests, a local inference API, a reproducible data snapshot and this report. Assignment 2 extends the same problem, features and split with tracked experiments, model registry and Airflow orchestration. Production scale, reliability and causal business goals below are design assumptions; measured offline results are separately identified.

Design decision	Selected approach
ML task	Binary churn classification used for ranking
Primary consumer	Retention agents working a capped review queue
Initial model	Logistic regression with train-only preprocessing
Serving pattern	Daily batch first; HTTP API as a local demonstrator
Release policy	Data and model gates, human approval for production
Outcome validation	Offline holdout, then prospective randomized pilot

1. Problem definition

1.1 Context, current situation and measurable problem

Assume an operator with 100,000 active consumer accounts, monthly billing and an existing CRM. The illustrative current process exports customers on month-to-month contracts and sorts them by monthly charge. It is transparent but cannot systematically combine tenure, service configuration, billing patterns and contract type. Its actual precision, cost and conversion rate are unknown because the hypothetical organization has not provided operational records. The first deployment milestone therefore includes instrumenting this rule baseline, rather than inventing a historical success rate.

The operational problem is to allocate a daily contact budget equal to at most 20% of an eligible scoring cohort. A useful model should concentrate more eventual churners in that queue than random selection and the incumbent rule, without overwhelming agents or triggering unsolicited contact. The scientific problem is to estimate $P(\text{churn in the following 30 days} \mid \text{information available at snapshot time})$. The sample dataset only supports an approximation to that task using its existing label.

1.2 Why ML is appropriate

Retention risk can depend on interacting signals: a short-tenure customer on an expensive month-to-month fiber plan may behave differently from a long-tenure customer with the same charge. A learned classifier can combine these signals and produce a continuous ordering, while a static rule requires manual combinations. Logistic regression is a defensible starting point because it is fast, interpretable and suitable for this modest tabular dataset. Its coefficients indicate conditional associations, not causes or personalized treatment effects.

ML would not be justified if features are unavailable before churn, if labels cannot be defined consistently, or if an evaluated rules policy yields comparable business outcomes at lower operating cost. Deployment is conditional on passing these checks. No customer is denied service, charged differently or automatically sent an offer solely because of a score.

1.3 Stakeholders and conflicting concerns

Stakeholder	Goal and concern	System response
Retention agents	Useful queue; limited time; understandable reasons	Capped queue, reason review, action history
Customers	Relevant assistance, privacy, no repeated unwanted contact	Opt-out and contact-frequency controls
Business owner	Incremental retained margin exceeds campaign cost	Randomized pilot and net-value KPI
Data science team	Valid labels, robust evaluation, reproducibility	Frozen partitions, lineage, model registry
Operations team	Predictable jobs, recoverability, manageable cost	Batch SLA, alerts, rollback and ownership
Privacy/compliance owner	Data minimization and auditable access	Restricted identifiers, retention limits, access logs

2. Functional requirements and contracts

The system ingests one record per eligible customer snapshot, validates it, assigns a risk score and publishes a ranked review list. Predictions assist an agent; campaign delivery remains the responsibility of the CRM. Production eligibility filters must remove opt-outs, customers already contacted within 30 days and accounts that have already terminated. These fields are absent from the sample and are requirements for integration, not implemented consent controls.

ID	Functional requirement	Acceptance condition
F1	Ingest a versioned customer snapshot	Checksum and schema report recorded for every batch
F2	Validate before scoring or training	Empty, duplicate, malformed or out-of-domain batches rejected
F3	Produce a churn probability per eligible customer	Finite score in [0,1]; customer key preserved
F4	Rank within a fixed capacity	Contact queue limited to $\text{ceil}(0.20 \times \text{eligible cohort})$
F5	Integrate with CRM batch import	Customer key, score, model version, snapshot date and rank
F6	Support agent oversight	Agent can defer, record action and submit reason for correction
F7	Retrain and evaluate models	Baseline comparison and quality gate before release
F8	Trace outcomes	Prediction/action/outcome joined by authorized pseudonymous key

2.1 Input specification

The implemented predictor contract has nine fields: tenure in months; MonthlyCharges and TotalCharges in source currency units; Contract; InternetService; PaymentMethod; PaperlessBilling; OnlineSecurity; and TechSupport. customerID is required for mapping and uniqueness but never becomes a predictor. Churn is required only for labeled training/evaluation batches. Demographic columns gender, SeniorCitizen, Partner and Dependents are excluded to reduce unnecessary use of sensitive/proxy features; exclusion alone does not guarantee fairness.

Numerical inputs must be finite and nonnegative. Blank TotalCharges is permitted only when tenure is zero and is imputed from the training median. Categories are checked against the documented source vocabulary. Unknown categories fail the external contract instead of silently changing semantics. The underlying encoder also tolerates unseen values as a defensive implementation measure, but production schema changes still require review.

2.2 Output and user interaction

The local API accepts a JSON object with a records array (1-1,000 records). POST /predict returns customerID and churn_probability; malformed requests receive HTTP 422. GET /health reports readiness after model loading. The API is a local demonstrator bound to loopback in the provided commands; authentication, transport encryption and per-user access control belong to production ingress design. Production batch output adds version and freshness metadata, while the prototype stores its model lineage alongside artifacts.

3. Non-functional and data requirements

3.1 Operating objectives (proposed, not load-tested guarantees)

Dimension	Initial target	Design and verification approach
Batch latency	Score 100,000 eligible rows within 10 minutes	Vectorized prediction; benchmark at launch scale
API performance	p95 below 200 ms at 20 requests/s	Load test end-to-end separately from model timing
Freshness	Publish daily queue by 07:00 local time	Batch completion and source-arrival monitoring
Availability	99.5% monthly for the scoring endpoint	Redundant replicas; health/readiness and error budget
Scalability	Grow from 100,000 to 1 million accounts	Partition input batches; scale workers, preserve schema
Recoverability	Restore service within 1 hour	Last approved model, versioned exports and runbook
Maintainability	Weekly candidate training; monthly review	Shared preprocessing pipeline and documented contracts

For a 30-day month, 99.5% availability permits approximately 216 minutes of unavailable time. This objective has not been established by a short local demo. Production API replicas should load one approved artifact at startup, expose readiness only after successful load, and continue serving that artifact while a new candidate is assessed. Failed training must not disrupt inference.

If ingestion is late or validation fails, the batch is quarantined and the data owner is alerted. An approved previous score export may be used for at most 48 hours, with a visible stale flag and fresh eligibility checks. Beyond that interval the agent queue falls back to the documented manual policy. No fabricated scores are substituted. A rollback changes the approved model pointer to a known version and reruns a small smoke batch before wider use.

3.2 Sources, volume, quality and privacy

The prototype bundles the public IBM CSV and a manifest containing its URL and SHA-256. It has 7,043 rows, 21 source columns and 1,869 churners (26.54%). Eleven TotalCharges values are blank; all belong to zero-tenure customers. These facts are validated by execution. Only the nine selected predictors, customerID and label are retained after validation.

Production sources would be billing, service subscription and CRM event systems. Monthly snapshots require source timestamps, record ownership and a label maturation window of at least 30 days plus ingestion delay. At 100,000 accounts over 12 months, approximately 1.2 million customer-month records are expected; this is a sizing assumption. Deduplicate on customer plus snapshot date and split chronologically, with account-aware analysis where repeated customers span periods.

Identifiers should be pseudonymized before analytics; reidentification mappings stay in a restricted CRM service. Encrypt storage and transport, grant role-based access, log administrative reads and avoid raw customer data in application logs. Proposed retention is 13 months for feature snapshots and 90 days for detailed scoring logs, subject to legal review and deletion obligations. Training dataset deletion requests require lineage-based impact review. Fairness evaluation should be performed in a controlled audit dataset where collection and use of relevant attributes are authorized.

4. Goals, metrics and acceptance criteria

The hierarchy starts with retained value, continues through dependable delivery, and ends with predictive performance. Accuracy alone is unsuitable because a classifier that predicts no churn for everyone already achieves about 73.46% accuracy in this sample.

Level	Metric	Baseline and proposed acceptance
Business	Incremental retained customers per 1,000 eligible accounts	Randomized business-as-usual control; positive lower 95% CI
Business	Incremental retained contribution minus total campaign cost	Positive after contact, offer and servicing costs
Business	Complaint/opt-out rate	No material increase versus control; review agreed margin
System	Daily completion before 07:00	At least 99% of scheduled batches per month
System	Freshness and model availability	At most 24-hour normal score age; 99.5% API target
Model	Validation average precision (AP)	At least 0.60 and 0.05 above prior baseline
Model	Lift at top 20%	At least 2.0 relative to cohort prevalence
Model	ROC-AUC, Brier, F1 and recall	Diagnostic metrics; investigate deterioration, do not cherry-pick

AP summarizes precision over recall increments: $AP = \text{sum of (recall increment} \times \text{precision)}$. It is used as the primary offline selection metric because it evaluates ranking under class imbalance. Precision@20% is the fraction of true churners within the highest-risk $\text{ceil}(0.2N)$ customers. Recall@20% is the fraction of all churners found within that queue. Lift@20% divides precision@20% by the cohort churn prevalence. Report the actual selected count because rounding affects small batches.

F1, precision and recall use a fixed 0.5 probability threshold in this prototype. That threshold is a diagnostic convention and is not the contact-budget policy. The Brier score is the mean squared error of probabilities; lower is better, but a single scalar does not replace a calibration plot or calibration-by-segment checks. Top-budget ranking does not require probabilities to be perfectly calibrated; financial decision rules do.

4.1 Business measurement plan

Randomize eligible customers to model-assisted retention and business-as-usual policies, with comparable contact budgets. Pre-register retained-customer and net-margin outcomes at 30 and 60 days, analyze intent-to-treat and stratify by contract and tenure. Size the pilot using measured baseline churn and a chosen minimum detectable effect; the sample dataset cannot provide intervention response rates. A hypothetical calculation such as $20,000 \text{ contacts} \times \text{incremental save probability} \times \text{retained margin}$ must remain a scenario, never a claimed benefit. Risk lift alone does not measure incremental saves.

5. High-level architecture

PRODUCTION DESIGN | offline training and daily retention workflow

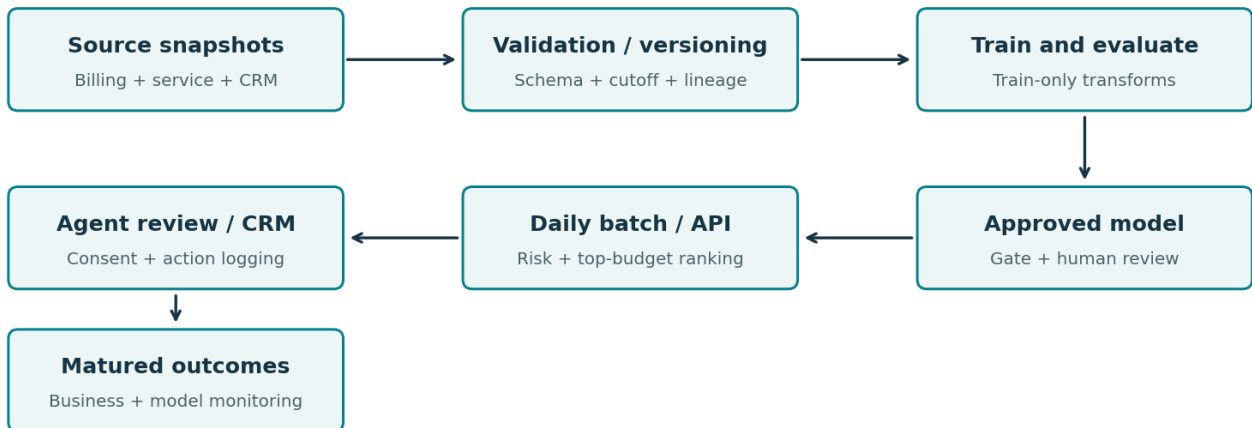


Figure 1. Proposed production architecture; feedback informs monitored retraining.

5.1 Data and control flow

The offline path starts with a dated source snapshot and immutable object storage. Validation produces a quality report, then partitioning separates development data from holdout data. A preprocessing/estimator pipeline fits only on training rows. Evaluation compares the candidate with an agreed baseline, and an approved registry version becomes eligible for serving. Training and inference use the same serialized transformations to avoid inconsistent feature encoding.

The daily scoring path reads eligible active customers, loads the approved pipeline, writes probabilities and ranks to a versioned export, then passes a capped review queue to the CRM. Prediction and intervention logs later join matured churn outcomes for monitoring. Labels and post-churn actions must never be fed into features for the same prediction time. Production keeps an explicit snapshot cutoff and event-time join rules.

Component	Responsibility	Technology considered and decision
Snapshot store	Immutable raw/curated datasets and manifests	Local CSV in demo; object storage in production
Validation layer	Schema, uniqueness, ranges and source checks	pandas contracts initially; scalable validator later
Training pipeline	Impute, scale, encode and fit one classifier	scikit-learn Pipeline and ColumnTransformer
Evaluation layer	Frozen splits, ranking metrics, acceptance gate	Python metrics and JSON/CSV evidence
Model store	Version, lineage, approvals, rollback reference	Local joblib baseline; MLflow in Assignment 2
Batch/API inference	Consistent scores from approved transforms	Python batch and FastAPI demonstrator
Orchestration	Dependencies, retries, schedule, run status	CLI baseline; Airflow in Assignment 2
Monitoring	Data freshness, latency, drift and label metrics	Structured logs initially; metrics platform at scale

5.2 Reliability, scale and lifecycle

The MVP deliberately avoids a separate feature store because the features are small, stable and computed in batch. An offline feature store becomes worthwhile when multiple teams reuse point-in-time features or ingestion semantics become more complex. A streaming architecture is unnecessary until experiments show that intraday freshness materially improves decisions.

Weekly retraining produces a candidate, not an automatic production replacement. Monitor daily missingness, category frequencies, row count and score distribution; evaluate AP, lift and calibration only after labels mature. Proposed alert thresholds include a greater-than-20% unexpected volume shift or AP decline above 0.05 over two matured cohorts. Investigate source changes and population shifts before retraining. These thresholds require tuning to normal seasonal variation.

6. Trade-offs and selected decisions

Trade-off	Alternatives and cost	Decision and reason to revisit
Accuracy versus latency	Ensembles capture nonlinearities but increase compute and explanation complexity	Start with logistic; compare ensembles in Assignment 2 and require meaningful gain
Freshness versus cost	Streaming improves recency but adds online joins, ordering and recovery complexity	Daily batch matches agent workflow; revisit only with measured intraday value
Simplicity versus performance	Independent feature services scale reuse but add dependencies	One serialized pipeline prevents skew; add feature store when reuse justifies it
Automation versus control	Fully automatic campaigns reduce effort but can make costly or unwanted contacts	Automate scores and validation, retain agent choice and production release approval
Privacy versus predictive breadth	Demographics may improve fit but raise collection and audit burdens	Use limited service/billing predictors; evaluate authorized fairness separately
Recall versus contact cost	Lower thresholds reach more churners but waste more contacts	Rank under fixed 20% budget; evaluate precision, recall and net value together

The decisions are linked. A batch architecture permits a simpler, lower-cost service and makes agent oversight practical. A high-capacity model is only justified if its incremental value survives a prospective pilot, not merely because it improves a metric in one random split. The current project intentionally exposes reproducible outputs and contracts before introducing distributed infrastructure.

7. Executed prototype and evaluation

Metric	Prior baseline (test)	Logistic C=1 (test)
average precision	0.2654	0.6249
roc auc	0.5000	0.8293
brier	0.1950	0.1426
f1	0.0000	0.5628
precision at capacity	0.2730	0.6312
recall at capacity	0.2059	0.4759
lift at capacity	1.0287	2.3780

Measured training time: 0.0256 seconds. Warm single-row model p95 latency: 5.93 ms (50 repetitions; excludes HTTP/network). Validation AP: 0.6737. These are local measurements, not throughput guarantees.

The holdout contains 1,409 customers. The 20% queue contains 282 records. At threshold 0.5, the confusion counts are TN=911, FP=124, FN=179, TP=195. Threshold classifications and the top-budget queue are different policies.

Execution evidence: Python 3.10.11; project-local environment; dependency, lint, test and full-data pipeline commands exited successfully. A real HTTP server returned 200 for valid prediction and 422 for an invalid empty batch. Machine-readable evidence and full test output are in evidence/verification.json and evidence/pytest.txt. Execution timestamp (UTC): 2026-10-03T16:46:28.967214+00:00.

7.1 Evaluation limitations and safeguards

Rows were split once using stratification and random seed 42: 4,225 train, 1,409 validation and 1,409 test. The median imputer, scaler and encoder fit inside the model pipeline using training rows only. The customer ID, target and unused demographics are excluded from predictors. This prevents several mechanical leakage paths but cannot correct unknown temporal relationships in the original source.

The prior classifier predicts the training churn prevalence for every row. Its AP equals evaluation prevalence and its ROC-AUC is 0.5. Its top-budget precision varies slightly with the deterministic tie ordering and should not be interpreted as learned ranking skill. The logistic baseline uses C=1 and maximum 2,000 optimization iterations. No hyperparameter search is performed for this first assignment.

Neither the local timing measurement nor a successful HTTP request establishes a production SLA. The experiment is a single split without a prospective cohort or causal intervention. Assignment 2 retains the partition contract so model improvements can be attributed to controlled experiment choices; because Assignment 1 already reports this holdout, it must not be treated as a newly unseen prospective dataset. Future production validation requires a fresh chronological cohort.

8. Submission evidence and reproduction

The submitted written report is DDM501_Assignment1_25ms13290_TrinhDucDuong.pdf. It covers the five assessed design areas in the assignment brief. The runnable prototype is supplementary evidence of feasibility; its measured offline performance does not replace the production design and trade-off analysis.

Repository: [ddm501-assignment-1-telco-churn-system-design](https://github.com/ddm501-assignment-1-telco-churn-system-design). The published [implementation snapshot 290f2fd](#) identifies the code, configuration, dataset and evidence accompanying this analysis. The report revision updates submission guidance and links; it does not change the trained baseline.

8.1 How to verify the prototype

Clone the repository and run the following commands from its root on Windows with Python 3.10 or 3.11. The setup script creates a project-local environment and installs the recorded dependencies. The verification script checks dependencies, lint, tests, full-data training and a real loopback HTTP request; subsequent runs can have different timings. The README also provides Linux and Docker instructions.

```
powershell -ExecutionPolicy Bypass -File setup.ps1
./venv/Scripts/python.exe scripts/verify.py
```

The [recorded test output](#) contains 12 passed tests with no failures. The [HTTP and environment verification](#) records a healthy service, HTTP 200 for valid predictions and HTTP 422 for invalid input. These are completed local checks, not claims about GitHub Actions or a deployed production service. No sibling assignment directory is required.

8.2 Assessment coverage

Assessed area	Weight	Location in this report
Problem definition	20%	Section 1: context, current process, measurable problem, ML rationale and stakeholders
Requirements analysis	20%	Sections 2-3: functionality, interfaces, performance, scaling, reliability, data and privacy
Goals and metrics	20%	Section 4: business/system/model hierarchy, baselines, thresholds and pilot design
High-level architecture	25%	Section 5 and Figure 1: flow, stages, responsibilities and technology choices
Trade-offs analysis	15%	Section 6: six explicit alternatives, costs and conditions for reconsideration

References and artifact map

[1] IBM. Telco customer churn sample repository and CSV.

<https://github.com/IBM/telco-customer-churn-on-icp4d> . Dataset snapshot retrieved 3 October 2026; source manifest and checksum are included in data/source.json. The IBM sample describes a fictional telecom company.

[2] scikit-learn. Common pitfalls and recommended practices: inconsistent preprocessing and data leakage. https://scikit-learn.org/stable/common_pitfalls.html

[3] scikit-learn. Average precision score and model evaluation.

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html

[4] Google. Rules of Machine Learning: practical production guidance.

<https://developers.google.com/machine-learning/guides/rules-of-ml>

[5] Sculley et al. Hidden Technical Debt in Machine Learning Systems. NeurIPS 2015.

<https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems>

Artifact	Purpose
churn/data.py; churn/model.py	Input contract, fixed split and train-only transformations
churn/pipeline.py; churn/api.py	Baseline execution and local HTTP service
config.yaml; requirements.txt	Configuration and independent runtime dependencies
tests/	Data, leakage, metric and integration checks
artifacts/summary.json	Measured baseline and holdout metrics
artifacts/split_ids.json	Auditable membership of all three partitions
evidence/	Test and actual execution records
README.md; setup.ps1; run.ps1	Standalone reproduction instructions

This is an independently runnable submission snapshot. All code, data and dependencies are resolved within this project; no sibling project is required. Author: Trịnh Đức Dương, student ID 25ms13290.